Lecture_4

Created by Dr. Preeti Jagadev

Using rbinom()

R has the capability to generate random numbers in a variety of configurations. Each configuration of random numbers reflects a specific underlying distribution. When working with events that have two states, such as heads and tails, the best distribution to use is the binomial distribution. In R, the "rbinom()" command generates random trials of the binomial distribution.Try this command:

```
rbinom(n=8, size=1, prob=0.5)
```

```
## [1] 1 0 0 0 0 0 0 0
```

What does it do?

n = 8: Generate 8 values.

size = 1: Each value is the result of 1 trial (like flipping 1 coin).

prob = 0.5: The chance of getting a 1 (success) is 50%.

Each value will be either: 1 = success (e.g., heads), 0 = failure (e.g., tails).

Label Results with factor()

```
set.seed(10)
rbinom(n=8,size=1,prob=0.5)
```

```
## [1] 1 0 0 1 0 0 0 0
```

```
factor(rbinom(n=8,size=1,prob=0.5),labels=c("red","blue"))
```

```
## [1] blue red  blue blue red  blue red  red
## Levels: red blue
```

set.seed(10): This line sets the seed for R's random number generator, so that the random results are repeatable. If someone else runs the same code, they will get the exact same output.

rbinom(n=8, size=1, prob=0.5): This simulates 8 coin flips, each with a 50% chance of success (i.e., a 1), like shown before.

factor(rbinom(n=8, size=1, prob=0.5), labels = c("red", "blue")): This line does another set of 8 coin flips, then replaces the numbers (0 and 1) with labels: 0 → "red", 1 → "blue"

So if the random values were: 0 0 1 1 0 1 0 0

You would get: red red blue blue red blue red red

Levels: red blue It turns numeric outcomes into category names (called a factor in R).

```
table(factor(rbinom(n=8, size=1, prob=0.5), labels=c("red","blue")))
```

```
##
##  red blue
```

```
##    4    4
```

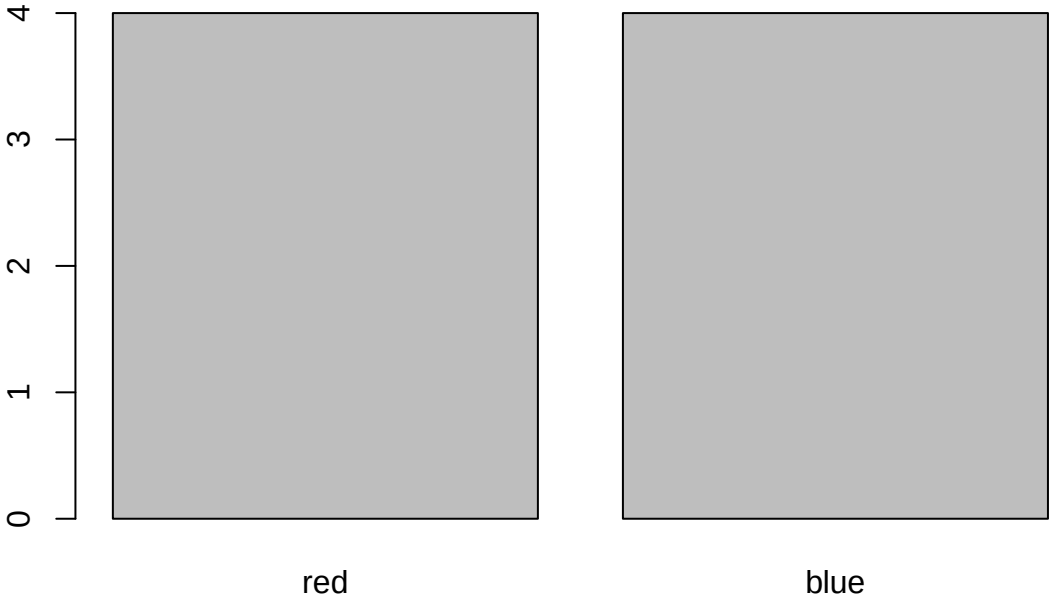
The table() function counts how many times each category appears.

So the output means:

4 outcomes were 0 (red), 4 outcomes were 1 (blue)

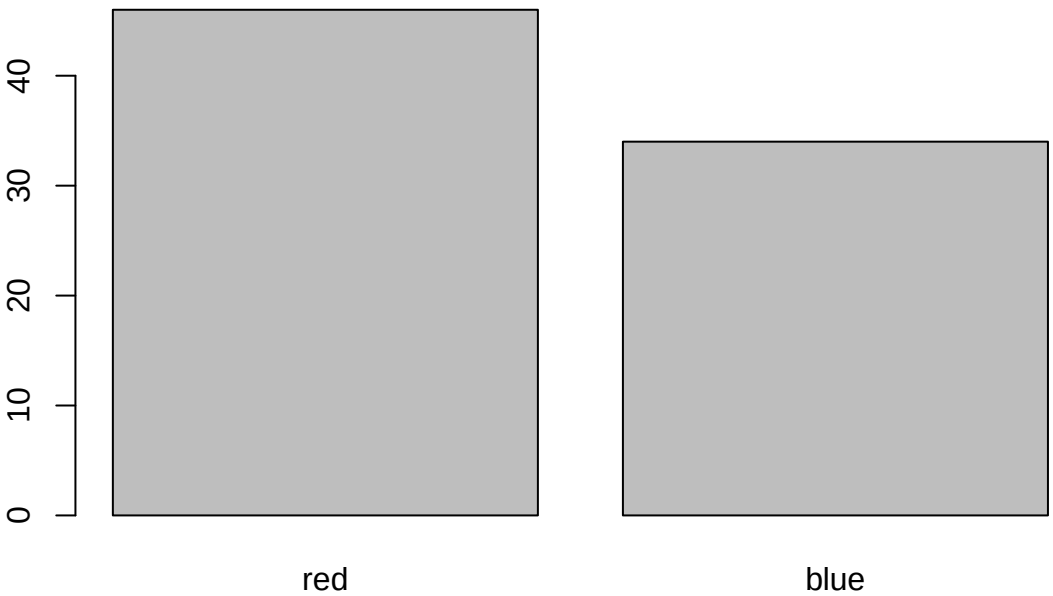
Tabulate and Plot:

```r
barplot(table(factor(
    rbinom(n=8,size=1,prob=0.5),
             labels=c("red","blue"))))
```



More Trials

```r
barplot(table(factor(
    rbinom(n=80,size=1,prob=0.5),
             labels=c("red","blue"))))
```



Samples States:

```
stateDF <- data.frame(
  state = state.name,
  population = state.x77[,"Population"])
sample(stateDF$population, size=6, replace=TRUE)
```

```
## [1]  2861 11197   746  4173  5313   590
```

state.name: This is a built-in vector in R with the names of all 50 U.S. states.

state.x77[,"Population"]: This grabs the Population column from the built-in dataset state.x77, which has numerical info about each U.S. state.

stateDF$population: This refers to the population column in the data frame called stateDF.

sample(..., size=6, ...): This says "Take a random sample of 6 values from that list."

replace=TRUE: This means you are allowed to pick the same value more than once.

```
mean(sample(stateDF$population, size=6, replace=TRUE))
```

```
## [1] 3722.833
```

Replication

One Sample isn't the whole story. When we collect one sample like surveying 10 people or testing 6 bottles of water, we get just one possible outcome. But that one sample might not perfectly represent the whole population. To really understand how reliable or trustworthy one sample is, we imagine repeating the process: taking another sample, and another. Again and again, from the same population. Each sample will give a slightly different result because of random chance. By looking at many samples, you can start to see the pattern and understand what's normal and what's just luck.

```
replicate(4,mean(sample(stateDF$population, size=16,replace=TRUE)),simplify=TRUE)
```

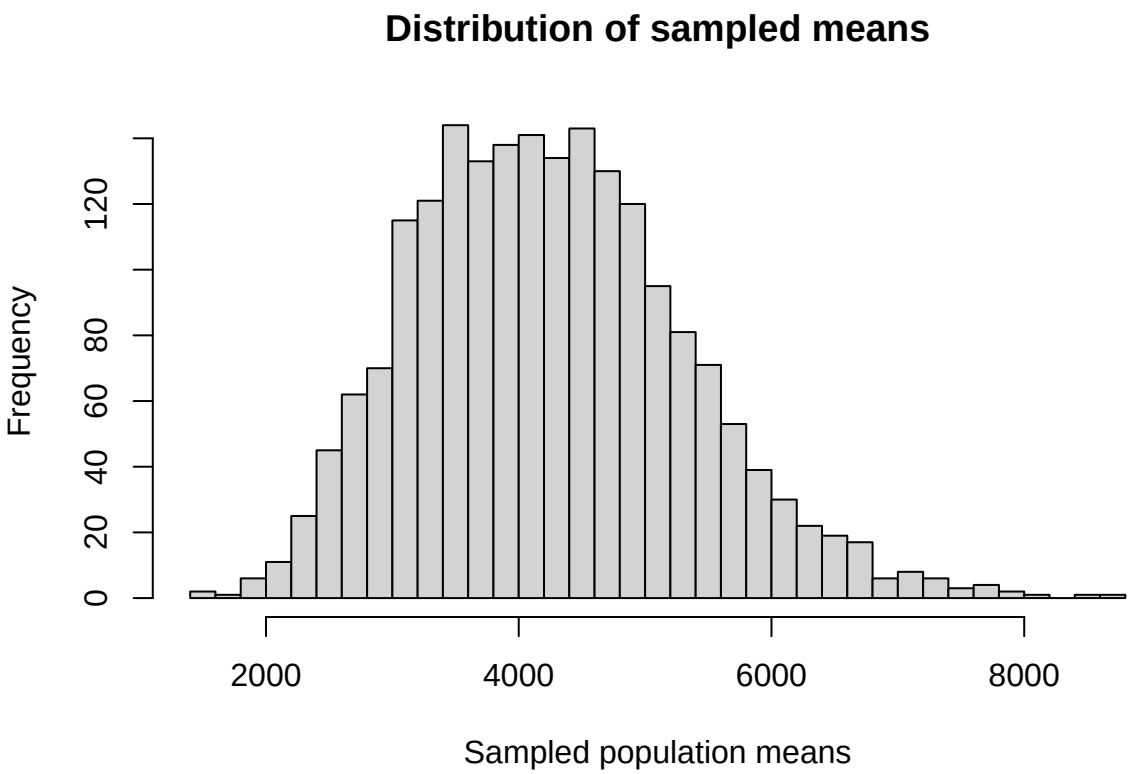
```
## [1] 5596.438 5325.312 3489.875 3046.000
```

Step-by-step Explanation:

1. sample(stateDF$population, size=16, replace=TRUE): Randomly selects 16 population values from the full list (with possible repeats, because replace=TRUE).
2. mean(…): Calculates the average of those 16 numbers.
3. replicate(4, …): Repeats this process 4 times, giving you 4 sample means.
4. simplify=TRUE: Returns the result as a vector of 4 numbers, not a matrix or list.

Finding the Histogram

```
hist(replicate(2000, mean(sample(stateDF$population,
                 size=16,replace=TRUE)),simplify=TRUE),
       breaks=30,
       main ="Distribution of sampled means",
       xlab="Sampled population means")
```



Features of this histogram:

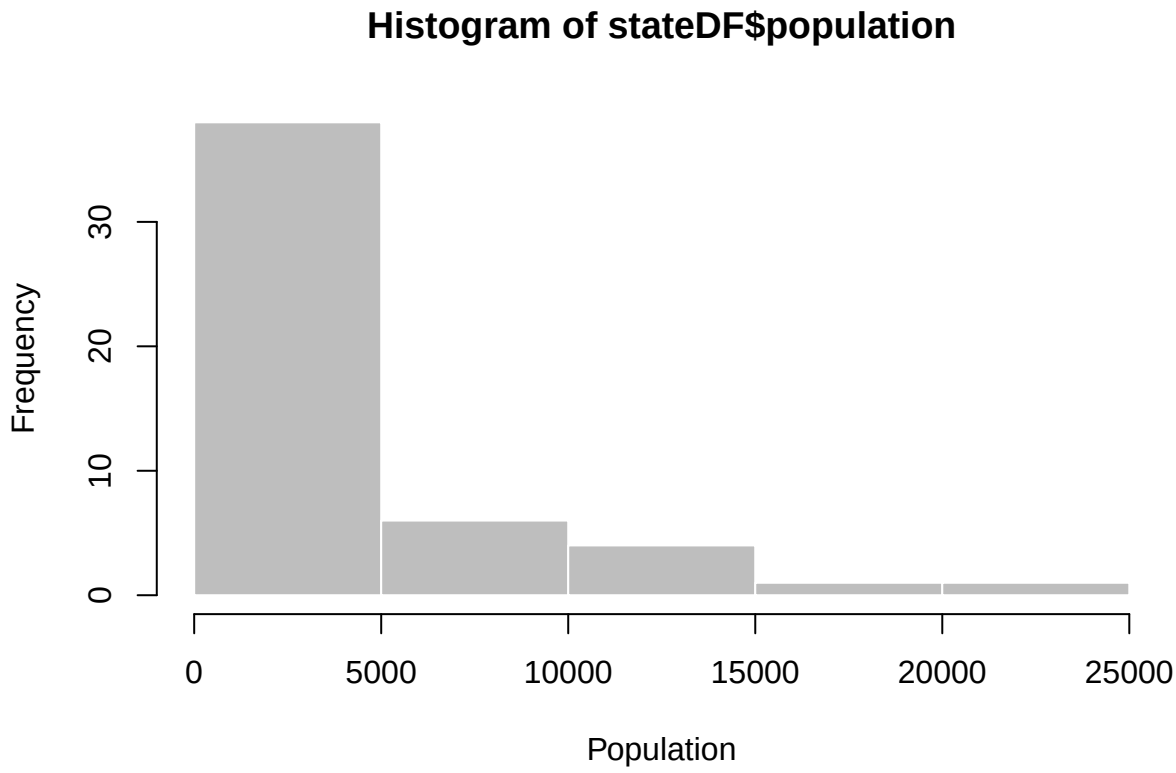
The data plotted consists of 2,000 means sampled from the census, where each sample is n=16 observations. The mode (the highest bar) is right near 4000. The distribution is forming a symmetric bell shape.

Important Note:

Histogram of Raw Data (stateDF$population):

This shows the actual populations of all 50 U.S. states. It's long-tailed, wherein most states have populations around 4,000 (in thousands), but a few big states (like California, Texas) push the numbers way up. So it's not symmetric, it's skewed to the right.

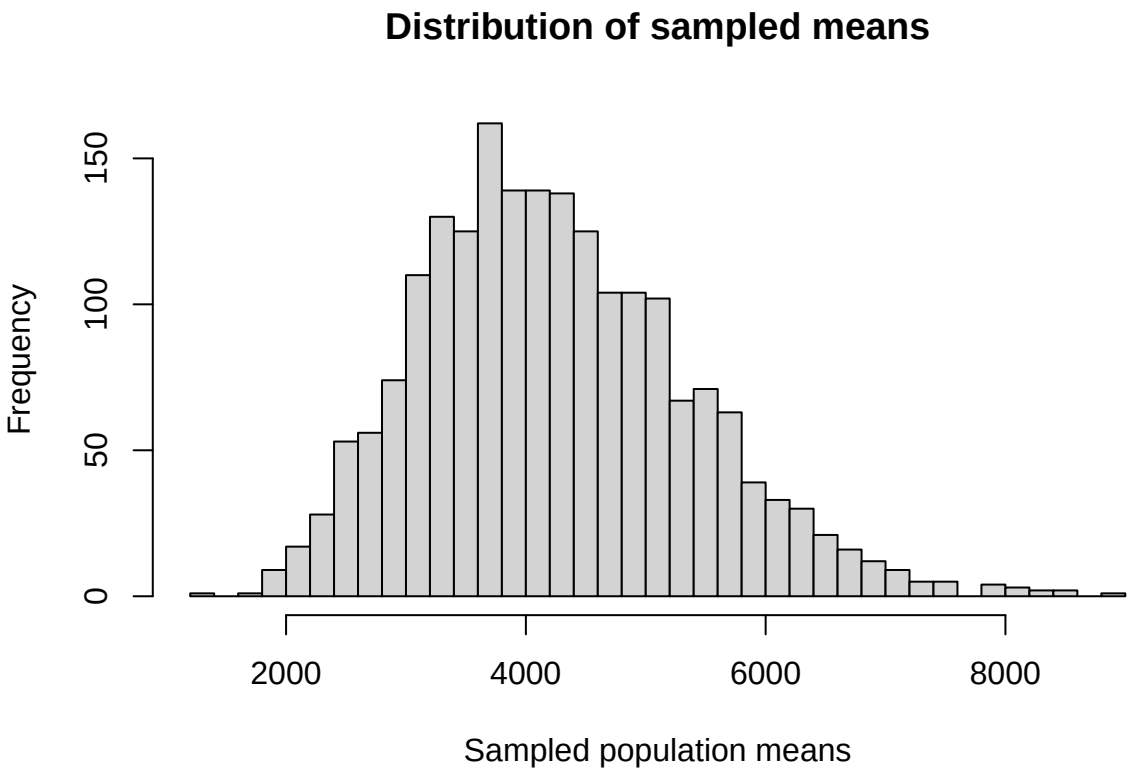
```r
hist(stateDF$population,
     main = "Histogram of stateDF$population",
     xlab = "Population",
     col = "gray",
     border = "white")
```



##

Histogram of Sample Means:

```r
hist(replicate(2000, mean(sample(stateDF$population,
                size=16,replace=TRUE)),simplify=TRUE),
       breaks=30,
       main ="Distribution of sampled means",
       xlab="Sampled population means")
```



This shows the means (averages) of 2,000 random samples (each of 16 states). It forms a bell-shaped curve, more symmetric.

Why Are the Shapes So Different?

Raw data is more spread out and uneven.Some states have tiny populations; others are massive. So the raw data has a long tail and is not balanced.

Sample means "smooth things out." When you average 16 states at a time, the extremes get balanced.High values (like California) and low ones (like Wyoming) cancel each other a bit. That's why sample means tend to pile up near the middle.

This is a visual way to understand the Central Limit Theorem:

The idea that averages from random samples will form a normal (bell-shaped) distribution, even if the original data is not bell-shaped.

Quantile command in R:

```
quantile(0:200,probs=c(0.25,0.75))
```

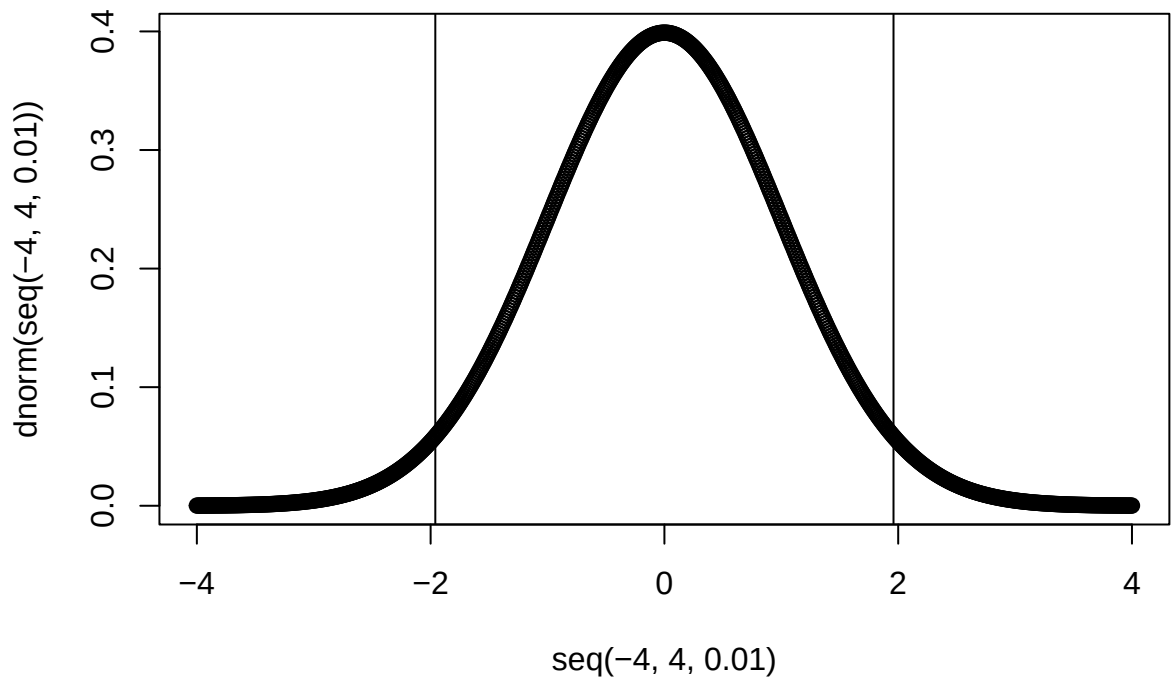
```
## 25% 75%
##  50 150
```

This command looks for the 0.25 and 0.75 quantiles within the digits 0 through 200.

Marking the Curve:

```
plot(seq(-4,4,.01),dnorm(seq(-4,4,.01)))
abline(v=qnorm(0.975))
```

```r
abline(v=qnorm(0.025))
```



Goal of This Code:

To visualize a normal distribution (bell curve) and show the middle 95% region of the curve using vertical lines at two key points: One at the 2.5% point (left tail), and one at the 97.5% point (right tail). This leaves 95% of the area in the middle between these two lines.

The Normal Curve

A normal curve (or bell curve) represents how values are distributed in many natural situations (like heights, test scores, etc.). It's symmetrical and shaped like a bell.

What the code does:

seq(-4, 4, 0.01): Generates x-values from -4 to +4 in small steps.

dnorm(…): Calculates the height (density) of the curve at each x-value.

plot(…): Draws the bell-shaped curve.

Drawing the Central 95% Region:

We want to draw vertical lines that show the boundaries of the middle 95% of the curve.

qnorm(0.975): Gives the x-value where 97.5% of the data lies to the left. That's about +1.96.

qnorm(0.025): Gives the x-value where only 2.5% lies to the left. That's about –1.96.

So, between –1.96 and +1.96 lies 95% of the total area under the curve (the part we care about).

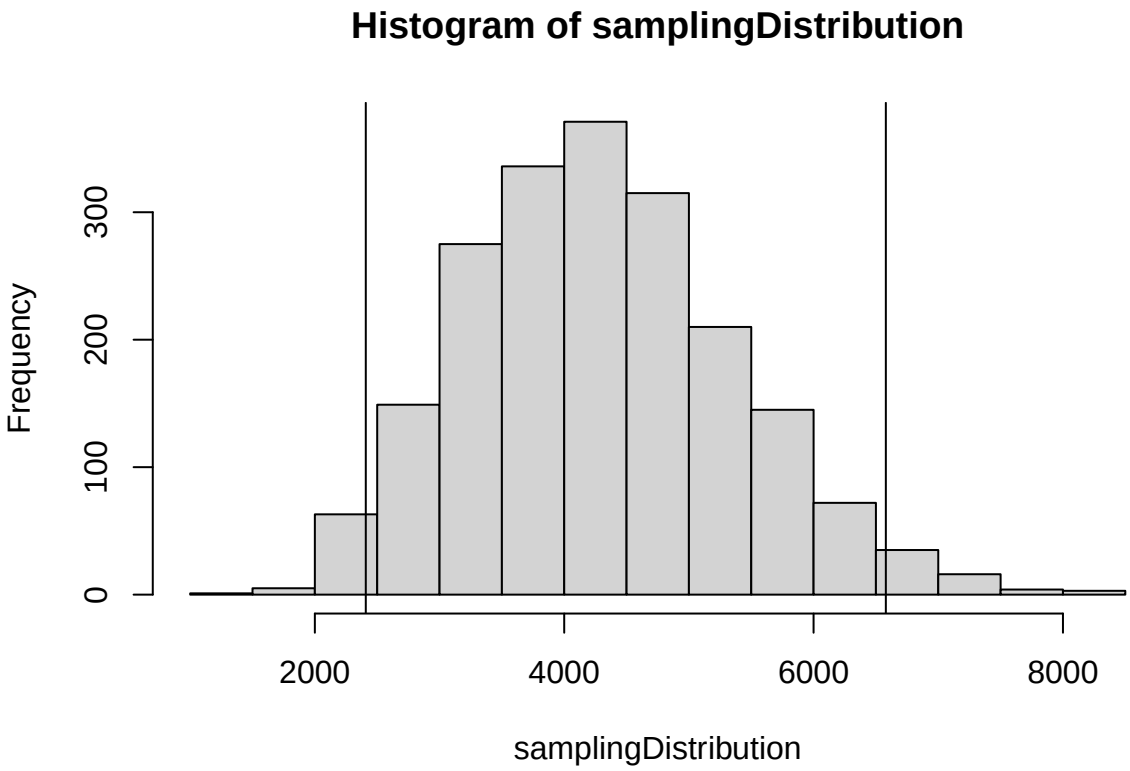
abline(v = …): This draws a vertical line at that specific x-value.

So we draw: One line at –1.96, and one line at +1.96

These two lines mark the edges of the middle 95% of the curve.

Now With Real Data:

```
samplingDistribution <- replicate(2000, mean(sample(stateDF$population, size=16,replace=TRUE)))
hist(samplingDistribution)
abline(v=quantile(samplingDistribution,0.025))
abline(v=quantile(samplingDistribution,0.975))
```



Step-by-step explanation of the code:

sample(stateDF$population, size = 16, replace = TRUE): This picks 16 states at random (with replacement) from the population column.

mean(…): It computes the mean population of those 16 states.

replicate(2000, …): This repeats the process 2000 times to create a sampling distribution of 2000 sample means.

hist(…): It plots a histogram of these 2000 means.

abline(v = quantile(…)): These draw vertical lines at the 2.5th percentile and the 97.5th percentile, meaning 95% of sample means lie between these two values.

What the graph shows:

A bell-shaped histogram of 2000 sample means.

Two vertical lines marking the lower 2.5% point and the upper 2.5% point.

So, the middle 95% of values lies between those two lines.

Now, let's answer the questions:

1. What percentage of all data points fall above 122?

Answer: You would visually inspect the histogram or use: mean(samplingDistribution > 122). If the histogram shows that almost no bars extend above 122, then less than 1% fall above that point. Thus, the answer is: Approximately 1% or less fall above 122.

2. Where is the median?

Answer: The median is the middle value—50% of points are below it. Use this code to find it: median(samplingDistribution). There's no line on the graph marking it, but it's roughly at the center of the histogram.

3. What percentage of points fall below the median?

Answer: Exactly 50%. That's the definition of the median.

Exploring the Mystery Sample:

```
MysterySample <- c(333, 1088, 3699, 3805,  3135, 3390, 99, 1049, 235, 129, 4299, 3649, 2295, 4663)
mean(MysterySample)
```

```
## [1] 2276.286
```

We are now using the mean of a mystery sample to figure out if it might come from the same population as the one we saw earlier (with the sampling distribution centered around ~4,000). Let's explore this step by step:

What do we know?

The sampling distribution of means (from the earlier histogram) had: Mean (center) 4,000

95% of sample means fall between 2,391 and 6,592.

Now, the main question:

Is the MysterySample from the same population?

Let's ask ourselves:

1. Does 2,276 fall within the usual range of sample means (2,391–6,592)?

Answer: No, it's below the 2.5% threshold (2,391) of the sampling distribution.

2. So what does that tell us?

Answer: The samples drawn from that population should give us sample means between 2,391 and 6,592. The MysterySample gave us a mean of 2,276, which is outside that fingerprint zone. Thus, it's unlikely that this sample came from the same population as the one that gave us the original sampling distribution. So this sample probably came from a different population, or from a very unusual subgroup of the same population.

Statistical Inference in a Nutshell:

Here's the basis for all statistical inference:

1. Construct a comparison distribution.
2. Identify a central zone containing 95% of the distribution: everything outside of this is considered an extreme value.
3. Compare a summary (e.g., mean) of a new sample of data to the distribution.

4. If the new sample falls in the "extreme zone," tentatively conclude that the new sample was obtained from some other source than the comparison distribution.